

Backend API Documentation - Postman Testing

Base URL

http://localhost:3000

Authentication

1. Login

Endpoint

POST /auth/login

Full URL

http://localhost:3000/auth/login

Body (JSON)

```
{
  "username": "admin",
  "password": "123456"
}
```

Response jika Multi Role

```
{
  "message": "Pilih role",
  "user_id": 1,
  "roles": [
    {
      "id": 1,
      "name": "Admin"
    },
    {
      "id": 2,
```

```
    "name": "Manager"
  }
]
}
```

Response Jika Single Role

```
{
  "token": "JWT_TOKEN"
}
```

2. Select Role

Digunakan jika user memiliki lebih dari 1 role.

Endpoint

```
POST /auth/select-role
```

Full URL

```
http://localhost:3000/auth/select-role
```

Body

```
{
  "user_id": 1,
  "role_id": 1
}
```

Response

```
{
  "token": "JWT_TOKEN"
}
```

Authorization Header

Untuk endpoint protected gunakan:

```
Authorization: Bearer JWT_TOKEN
```

Contoh:

```
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

USER CRUD

1. Get All Users

Endpoint

```
GET /users
```

Full URL

```
http://localhost:3000/users
```

Headers

```
Authorization: Bearer JWT_TOKEN
```

Response

```
[
  {
    "id": 1,
    "username": "admin"
  },
  {
    "id": 2,
    "username": "budi"
  }
]
```

```
}  
]
```

2. Get User By ID

Endpoint

```
GET /users/:id
```

Example

```
http://localhost:3000/users/1
```

Headers

```
Authorization: Bearer JWT_TOKEN
```

Response

```
{  
  "id": 1,  
  "username": "admin"  
}
```

3. Create User

Endpoint

```
POST /users
```

Full URL

```
http://localhost:3000/users
```

Headers

```
Authorization: Bearer JWT_TOKEN  
Content-Type: application/json
```

Body

```
{  
  "username": "andi",  
  "password": "123456"  
}
```

Response

```
{  
  "message": "User berhasil dibuat",  
  "data": {  
    "id": 3,  
    "username": "andi"  
  }  
}
```

4. Update User

Endpoint

```
PUT /users/:id
```

Example

```
http://localhost:3000/users/3
```

Headers

```
Authorization: Bearer JWT_TOKEN  
Content-Type: application/json
```

Body

```
{
  "username": "andi_update"
}
```

Response

```
{
  "message": "User berhasil diupdate",
  "data": {
    "id": 3,
    "username": "andi_update"
  }
}
```

5. Delete User

Endpoint

```
DELETE /users/:id
```

Example

```
http://localhost:3000/users/3
```

Headers

```
Authorization: Bearer JWT_TOKEN
```

Response

```
{
  "message": "User berhasil dihapus"
}
```

MENU CRUD

1. Get Menus

Endpoint

```
GET /menus
```

Full URL

```
http://localhost:3000/menus
```

Headers

```
Authorization: Bearer JWT_TOKEN
```

Response

```
[
  {
    "id": 1,
    "name": "Dashboard",
    "path": "/dashboard",
    "parent_id": null,
    "menu_order": 1,
    "children": []
  }
]
```

2. Create Menu

Endpoint

```
POST /menus
```

Full URL

```
http://localhost:3000/menus
```

Headers

```
Authorization: Bearer JWT_TOKEN  
Content-Type: application/json
```

Body

```
{  
  "name": "Dashboard",  
  "path": "/dashboard",  
  "parent_id": null,  
  "menu_order": 1  
}
```

Response

```
{  
  "message": "Menu berhasil dibuat",  
  "data": {  
    "id": 1,  
    "name": "Dashboard",  
    "path": "/dashboard",  
    "parent_id": null,  
    "menu_order": 1  
  }  
}
```

3. Update Menu

Endpoint

```
PUT /menus/:id
```


Example

```
http://localhost:3000/menus/1
```

Headers

```
Authorization: Bearer JWT_TOKEN  
Content-Type: application/json
```

Body

```
{  
  "name": "Dashboard Admin",  
  "path": "/dashboard",  
  "parent_id": null,  
  "menu_order": 1  
}
```

Response

```
{  
  "message": "Menu berhasil diupdate",  
  "data": {  
    "id": 1,  
    "name": "Dashboard Admin",  
    "path": "/dashboard",  
    "parent_id": null,  
    "menu_order": 1  
  }  
}
```

4. Delete Menu

Endpoint

```
DELETE /menus/:id
```

Example

```
http://localhost:3000/menus/1
```

Headers

```
Authorization: Bearer JWT_TOKEN
```

Response

```
{  
  "message": "Menu berhasil dihapus"  
}
```

Role Middleware Testing

Test Forbidden Access

Login menggunakan role selain admin lalu coba:

```
POST /users
```

atau:

```
DELETE /users/1
```

Expected Response

```
{  
  "message": "Forbidden Access"  
}
```

Database Tables

users

Menyimpan data user.

roles

Menyimpan data role.

user_roles

Relasi many-to-many user dan role.

menus

Menyimpan data menu.

role_menus

Relasi role dan menu.

Tech Stack

- Node.js
 - Express.js
 - PostgreSQL
 - JWT Authentication
 - Role Based Access Control (RBAC)
 - bcrypt
-

Features

- JWT Authentication
- Multi Role Login
- Role Middleware
- User CRUD
- Menu CRUD
- Dynamic Menu By Role
- Protected Routes

- PostgreSQL Integration